

YaPcore Admin Dashboard

Browser-based **admin control panel** for headless hosts and remote operators. Set up the network, configure YaP Link, manage plugins, and monitor health — no SSH required for day-to-day ops.

Controls the **YaPcore chassis** in front of **YaP-Folia** (game child JVM). Build with `./scripts/build-yap-folia.sh`.

Modern **sidebar shell** (Overview · Server · People · Content · Gameplay) with dark theme, page search, stat cards, and full plugin config editors where the backend supports it.

Colored text (ranks, MOTD, tab list, NPC names, chat format) uses **clickable Minecraft color swatches**, a custom hex picker, and bold/italic controls — plus a live preview. Hex is stored as `&#rrggbb`.

Enable

Default on. Config (`config/server.properties`):

```
web-dashboard-enabled=true
web-dashboard-port=8080
web-dashboard-bind=0.0.0.0
web-dashboard-token=
web-dashboard-localhost-only=false
ops=
auto-op=false
```

Empty `web-dashboard-token` → a random token is generated on first start and saved into the config file (also printed in the server log).

Open

```
http://127.0.0.1:8080/
```

Paste the token on the login screen (or set `Authorization: Bearer <token>`).

Quick access: type `dashboard` in the YaP Control Panel console or headless stdin — prints a one-click login URL and token. In the Swing GUI, use **Web Dashboard** (header) or **Connect → Open**.

Login links include `?token=...` so the browser signs in automatically (token is stripped from the address bar after load).

Navigation

Group	Tabs
Overview	Dashboard (status), Network setup, Connect
Server	Console, Server setup, YaP Link
People	Players, Access & ranks, Rank pack
Content	Plugins, Modules, Packs, World, Regions, NPCs
Gameplay	Essentials, Vehicles, Pregen, Player data, Chat, Tab list, MMO , Map, Guard, Protect, Discord

Static assets: `src/main/resources/web/` — `app-shell.js` , `app-core.js` , `app-*-panels.js` , `style.css` .

Admin tab (infrastructure)

`/api/admin` — operator setup without editing files by hand:

Section	What you configure
Dashboard access	Port, bind, localhost-only, enable/disable, rotate token
External access	Internet expose, domain, public Java/Bedrock/pack ports, DNS SRV hint
nginx	Localhost assist, stream/HTTP ports, domain, config dry-run
Proxy / authority	Velocity forwarding, Link embed mode, link home path

POST actions: `save-access` , `save-nginx` , `save-dashboard` , `save-proxy` , `rotate-token` , `nginx-dry-run` , `run-smoke` , `crashdump` .

All tabs & API routes

Tab	API	GET snapshot	POST actions (high level)
Dashboard	<code>/api/status</code>	running, heap, ticks, link process, network health	—
Connect	<code>/api/connect</code>	Java/Bedrock/crossplay join, pack URL	—
Console	<code>/api/console</code> , <code>/api/command</code>	log backlog	run command · SSE <code>/api/console/stream</code>
Server setup	<code>/api/config</code>	everyday server switches (name, who can join, RAM)	save config keys
YaP Link	<code>/api/link</code> , <code>/api/link/console</code>	proxy, backends, forced hosts, selector	start, stop, save-proxy, save-servers, command · Link SSE
Players	<code>/api/players</code>	online list, spawn, moderation flags	kick, ban, tempban, ipban, mute, warn, timeout, tp, <i>set-rank</i> , <i>promote</i> , <i>demote</i> , <i>eco give/take/set/reset*</i> , bal, history, check, banlist
Access & ranks	<code>/api/access</code>	ops, auto-op, default group, groups, tracks, permission catalog , group nodes	save-group-nodes, save-ops, save-auto-op, set-default-group, op, deop, set-group, promote, demote, group/user perm, dump, reload, applypack
Rank pack	<code>/api/ranks</code>	pack applied, auto-apply	apply, force, reset-marker, status
Plugins	<code>/api/plugins</code>	jar list + compat matrix	install, remove
Plugin settings	<code>/api/plugin-config</code>	first-party YAML with plain-language titles + Yes/No	save + reload, reload
Modules	<code>/api/modules</code>	module jars	install, remove
Packs	<code>/api/packs</code>	resource packs, active set	setActive, add, remove, clear
World	<code>/api/world</code>	schematics, brush max, load/unload flags	load, unload, reload, schem-list, save-brush
Regions	<code>/api/regions</code>	region table (JSON), flag names	define (cuboid coords), flag-set , list
NPCs	<code>/api/npcs</code>	npc table, quest ids	create, remove, setquest, setdialogue, respawn, reload, info

Tab	API	GET snapshot	POST actions (high level)
Essentials	/api/essentials	features, MOTD, rules, spawn	reload, broadcast, save-motd, save-rules, set-feature
Vehicles	/api/vehicles	type list	spawn, shop, list, types, upgrades
Pregen	/api/pregen	job status	start, pause, resume, cancel
Player data	/api/playerdata	economy, auth, feature toggles	reload, save, set-feature
Kits	/api/kits	kits.yml definitions, items, armor slots	save-kit, delete-kit, clone-kit , give, grant, reload
Tebex store	/api/tebex	jar present, secret masked, buy command, proxy, package recipes	set-secret, save-settings , reload, info, forcecheck
Chat	/api/chat	channels, slow mode, filter, relay	reload, clearchat, save-settings
Tab list	/api/tab	header/footer/sidebar/bossbar	save-header/footer/sidebar/settings/bossbar, reload
Map	/api/map	map URL, tiles, worlds, render interval	reload, render, save-settings
Guard	/api/guard	check toggles, kick threshold, decay, alerts	reload, player-status, save-settings
Protect	/api/protect	logging, retention, status	reload, prune, lookup, rollback, save-settings
MMO	/api/mmo	skills, abilities, hiscores, boss kills, combat bar bindings	reload-abilities , reload-mmo

Legacy routes (superseded by UI tabs): /api/moderation → **Players**; /api/perms → **Access & ranks**.

Pack HTTP stays on **:8081**. Dashboard is a separate port (**:8080**).

Access & ranks

Full operator control without /op and /yapperm by hand:

- **Minecraft OPs** — chip list, add/remove, persisted to server.properties
- **Auto-op** — toggle for first join
- **Default rank** — YaPPerms default group dropdown
- **Group cards** — click to edit tag, name color, and chat color with **color swatches + custom hex** (no & codes required), plus suffix, weight, and inheritance. Live chat preview updates as you pick colors.
- **Rank permissions** — catalog of player / staff / vanilla / Paper commands plus **nodes discovered from installed plugin.yml**; inherit · allow · deny; any custom / wildcard node (bulk add); saved with save-group-nodes
- **Create rank with a pack** — empty / player / staff / admin, or copy perms from an existing rank (template , cloneFrom on create-group)
- **Apply pack / copy perms** onto an existing rank (apply-template , clone-group)
- **Promote / demote** — track-based rank changes

POST /api/access {"action":"save-group-nodes","group":"vip","allow":"yapessentials.fly,...","deny":"minecraft.command.op","unset":"yapessentials.god"} writes plugins/YaPPerms/config.yml (starter-grants + editor-nodes) and applies live via yapperm editor-apply . Refresh dumps live extras with yapperm dump → editor-snapshot.yml .

Kits (yap-playerdata)

Gameplay → Kits builds claim kits in `plugins/YaPPlayerData/kits.yml` (same file `/createkit` uses):

- Cooldown, max uses, economy cost, first-join, console commands (`{player}`)
- Item rows: Bukkit material, amount, slot (inventory / armor / offhand), name, lore, enchantments (`sharpness:5`)
- Clone / delete; **Give now** (`kit give`) or **Grant** (`kit grant`) to a player
- Saves YAML then runs `yapdata reload` (YAML is kept if Folia is down)

GET/POST `/api/kits` — `save-kit`, `delete-kit`, `clone-kit`, `give`, `grant`, `reload`. Item lines in POST:

`MATERIAL|amount|slot|name|lore|enchants`. Players still need `yapdata.kit.<id>` (or `yapdata.kit.*`) on Access & ranks.

`/createkit` Bukkit stacks stay readable; saving from the dashboard writes the material form (NBT beyond name/lore/enchants is dropped).

Tebex store

Gameplay → Tebex store wires the GPLv3 Folia plugin (`plugins/tebex.jar`):

- Status: jar present, secret configured (masked), `/buy` command, proxy mode
- **Save secret** → writes `plugins/Tebex/config.yml` and runs `tebex secret <key>`
- Buy command / proxy / verbose toggles → `tebex reload`
- Copy-ready package recipes (`{username}`) for VIP rank and kit unlocks
- Links to creator.tebex.io and Tebex Minecraft docs

GET/POST `/api/tebex` — `set-secret`, `save-settings`, `reload`, `info`, `forcecheck`. Full guide: [TEBEX.md](#).

Players

Staff moderation panel: **online** table plus **everyone who has ever joined** (username, nickname, UUID, last IP, all known IPs, first/last seen). Click a row to kick/ban/mute/IP-ban. IP bans use the stored last IP after they leave.

Economy on the same panel: amount field + **Give money** / **Take money** / **Set balance** / **Reset** / **Check balance** (`eco ...` / `bal` via console).

Requires YaP-Folia running + `yap-moderation` / `yap-perms` / `yap-playerdata`. Snapshot is refreshed with `yapmod` seen `snapshot` when you hit Refresh.

MMO (`yap-skills`, `yap-abilities`, `yap-mmo-content`)

Gameplay → MMO tab — read-only progression overview plus ability hotbar status:

- Skill/content jar presence, ability catalog count, boss/area counts
- Dual hotbar + ability book flags (including Shift+F)
- Online players with combat bar bindings (keys 4–9)
- Hiscore preview + boss kill totals (live when Folia is running)

Action	POST <code>/api/mmo</code>
Reload ability YAML	<code>{"action":"reload-abilities"}</code>
Reload MMO content	<code>{"action":"reload-mmo"}</code>

Live data uses console exports: `yapmmo snapshot json`, `yapabilities snapshot json`.

NPCs (`yap-npcs`)

Dashboard drives the plugin directly:

- Create NPC at world coordinates (console: `npc create <id> at <world> <x> <y> <z> [yaw] [name]`)
- Edit display name, quest, dialogue; remove; respawn all; reload config
- GET `/api/npcs` returns structured `npcs[]` from `npc list json`

Regions (yap-regions)

- Define admin cuboids from the UI (console: `region define <name> at <world> x1 y1 z1 x2 y2 z2`)
- Set WorldGuard-class flags (pvp, build, entry, ...) allow/deny per region
- GET `/api/regions` returns structured `regions[]` from `region list json`

Chat, Guard, Protect, Map, World

These tabs **write plugin YAML** via `save-settings` (or equivalent) and reload the plugin — not just run one-off commands.

Network health (Dashboard tab)

`/api/status` includes `networkHealth` :

- Folia running, bedrock/crossplay/velocity flags
- Link process running (`linkProcessRunning`), config + suite completeness
- Plugin count + compat warning count
- **Ops plugins** — Phase 8 jar readiness (Protect, Chat, Moderation, Player data, Map, Discord) with one-line detail per plugin

Map tab

Serves tiles via YaPcore pack HTTP when `use-yapcore-server: true` (default). First render runs ~2s after plugin enable; full re-render on `render-interval-minutes`. Tune `sample-chunk-radius` and `max-height` on low-CPU hosts — see plugin `config.yml` comments.

Discord tab

Webhooks and relay toggles. Safe setup order documented in [DISCORD_RELAY.md](#). MC→Discord and Discord→MC stay **off** until webhooks and inbound secrets are set.

YaP Link tab (proxy process)

Separate from the main **Console** tab (YaPcore/Folia). Requires `link-embed=false` .

Settings UI — configure the full proxy without editing files by hand:

- **Proxy settings** — bind, MOTD, max players, online mode, public host/port, chat relay, bedrock block
- **Backends** — add/remove servers (`hub` , `survival` , ...), host:port, optional per-backend Bedrock address
- **Try order** — fallback order when a backend is down (e.g. `hub` , `survival`)
- **Forced hosts** — route by hostname (e.g. `hub.yourdomain.com` → `hub`)
- **Hub / server selector** — `yaplink-server-selector` `hub` + session lock

Saves write `link-data/link.properties` . If Link is running, the dashboard sends `reload` automatically.

Action	POST <code>/api/link</code> body
Start Link	<code>{"action":"start"}</code>
Stop Link	<code>{"action":"stop"}</code>
Run command	<code>{"action":"command","command":"reload"}</code>
Enable backend forwarding	<code>{"action":"enable-backend-forwarding"}</code>

Action	POST /api/link body
Save proxy settings	<code>{"action": "save-proxy", ...}</code>
Save backends	<code>{"action": "save-servers", "servers": [...], "try": [...], "forcedHosts": [...]}</code>
Save selector	<code>{"action": "save-selector", "hubServer": "hub", "sessionLock": "true"}</code>

Live log: **GET** `/api/link/console` · **SSE** `/api/link/console/stream?token=...`

Plugin compat (Plugins tab)

Each jar shows status from [PLUGIN_COMPAT_MATRIX.md](#): `native`, `works`, `broken`, `folia-build`, or `unknown`, plus native alternative hint.

Ranks via console / tab

```
gradle installProductDefaults
# after Folia/Paper is up:
ranks apply
/yapperm user Steve parent set vip
/promote Steve
```

Dashboard **Access & ranks** and **Rank pack** tabs call `/api/access` and `/api/ranks`. Reference: [examples/yapperms/ranks-reference.txt](#). Config: `plugins/YaPPerms/config.yml`. See [PERMISSIONS.md](#).

Optional: `yap-ranks-auto-apply=true` in `config/server.properties`.

Security

- Treat the token like a password.
- Prefer `web-dashboard-localhost-only=true` or bind to a private IP, then put nginx + TLS in front for public access.
- Do not expose `:8080` to the internet without auth + TLS.
- Use **Network setup** → **Rotate token** if the secret may have leaked.